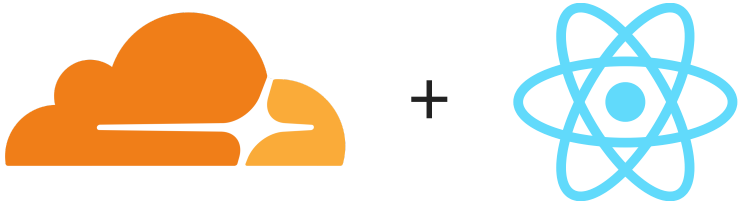
DEPLOY AND TEST FULL-STACK REACT APPS ON CLOUDFLARE



INSTRUCTORS

- Christian Sparks
- Dario Piotrowicz

CHRISTIAN SPARKS

- Builds and Automation team @ Cloudflare
- github.com/cmsparks
- linkedin.com/in/cmsparks

DARIO PIOTROWICZ

- Frameworks team @ Cloudflare
- github.com/dario-piotrowicz

WORKSHOP

Is about creating a simple full stack React application and deploy it to the Cloudflare platform

Alongside comprehensive testing 🧪

CLOUDFLARE WORKERS

Workers the Cloudflare's edge network runtime

Includes

- Static Assets hosting
- an API to access cloudflare resources (Bindings)
- and more... 🚀

PROJECT OVERVIEW

In this workshop we'll be creating a trading card generator app built using:

- the Remix React full-stack framework
- Cloudflare Workers
- Cloudflare Bindings, including
  - Workers KV to store the card's information
  - and Workers AI to generate the card's image

COMPLETED APPLICATION

Here's a sneak peek at the final product:

REQUIREMENTS

- Your favorite IDE
- `npm` (or your preferred package manager)
- `git` so that you can clone and interact with the workshop's git repository
- A Cloudflare account
- Enthusiasm! 🚀

GITHUB REPOSITORY

AGENDA

- Create Project (~15m)
- UI implementation (~30m)
- Setup workers code (~1h)
- E2E Implementation (~45m)
- End of workshop (~15m)

Total time: ~3h

REMIX

- Fullstack React framework
- Modern UX & DX
- Web standards based
- Soon to be React Router v7
- Alternatives: Next.js , tanstack, waku, @lazarv/react-server, etc...

REMIX + CLOUDFLARE

- Out of the box cloudflare support
- Polished APIs for accessing Cloudflare resources
- Started template included in C3

REMIX EXERCISE

Let's implement our application's UI form

`exercises/01-ui-card-form`

Git commit: `da77bbc`

DEPLOYING A WORKER

Now that we've set up our Remix application, let's try deploying it by running:

```
npx wrangler login
npm run deploy
```

USING WORKERS SERVICES

To generate and save trading cards in our fullstack app, we'll be using Cloudflare Workers with the following Workers bindings:

- Workers KV - A lightweight key value store, used to store our card metadata
- R2 - S3 Compatible object storage, used for our card image data
- Workers AI - API for inference related tasks, for image generation

WHAT ARE BINDINGS IN WORKERS?

Workers Bindings do two things:

- They grant capabilities/permissions (i.e. your worker has access to an R2 Bucket)
- They inject the service's API into your Worker's environment

BENEFITS OF BINDINGS

There are multiple benefits to this model:

- Lower security risk because you can't accidentally leak an API token
- No boilerplate needed to initialize your API client
- Easier to understand what resources are being used

EXERCISE: IMPLEMENTING THE CARDMANAGER CLASS

Let's implement our Workers code using the CardManager class

`exercises/02-business-logic`

Git commit: `a301b6b`

UI INTEGRATION

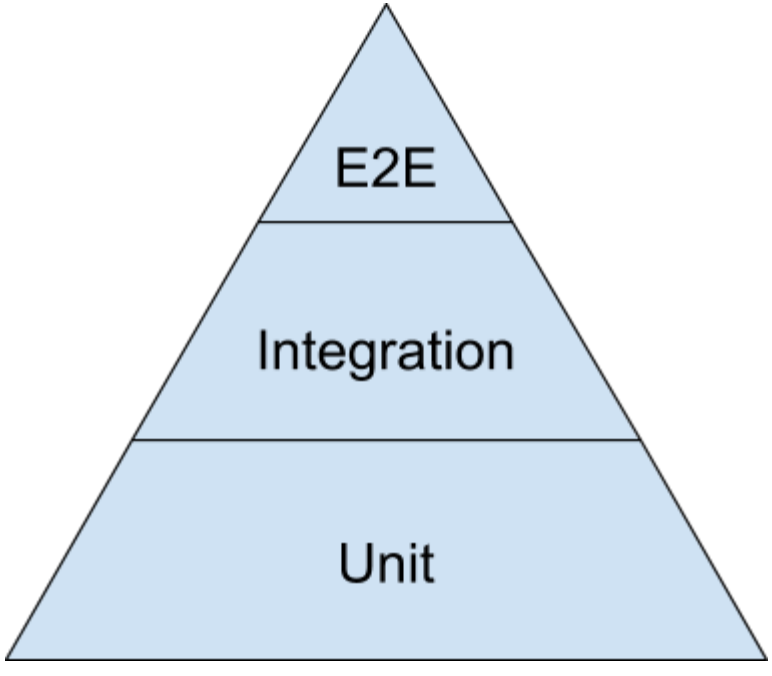
Let's now integrate our new backend/workers logic with our frontend/UI

`exercises/03-ui-integration`

Git commit: `291ecfb`

END-TO-END (E2E) TESTING

- Aims to test a system's overall behavior (against user expectations)
- More realistic than unit testing
- More expensive and time consuming


E2E
Integration
Unit

PLAYWRIGHT

- E2E Testing framework from Microsoft
- Fast and intuitive
- Many features such as auto reties, trace viewers, codegen, etc...
- Multi browser and multi languages support
- Alternatives: cypress, puppeteer, etc...

GETTING STARTED WITH PLAYWRIGHT

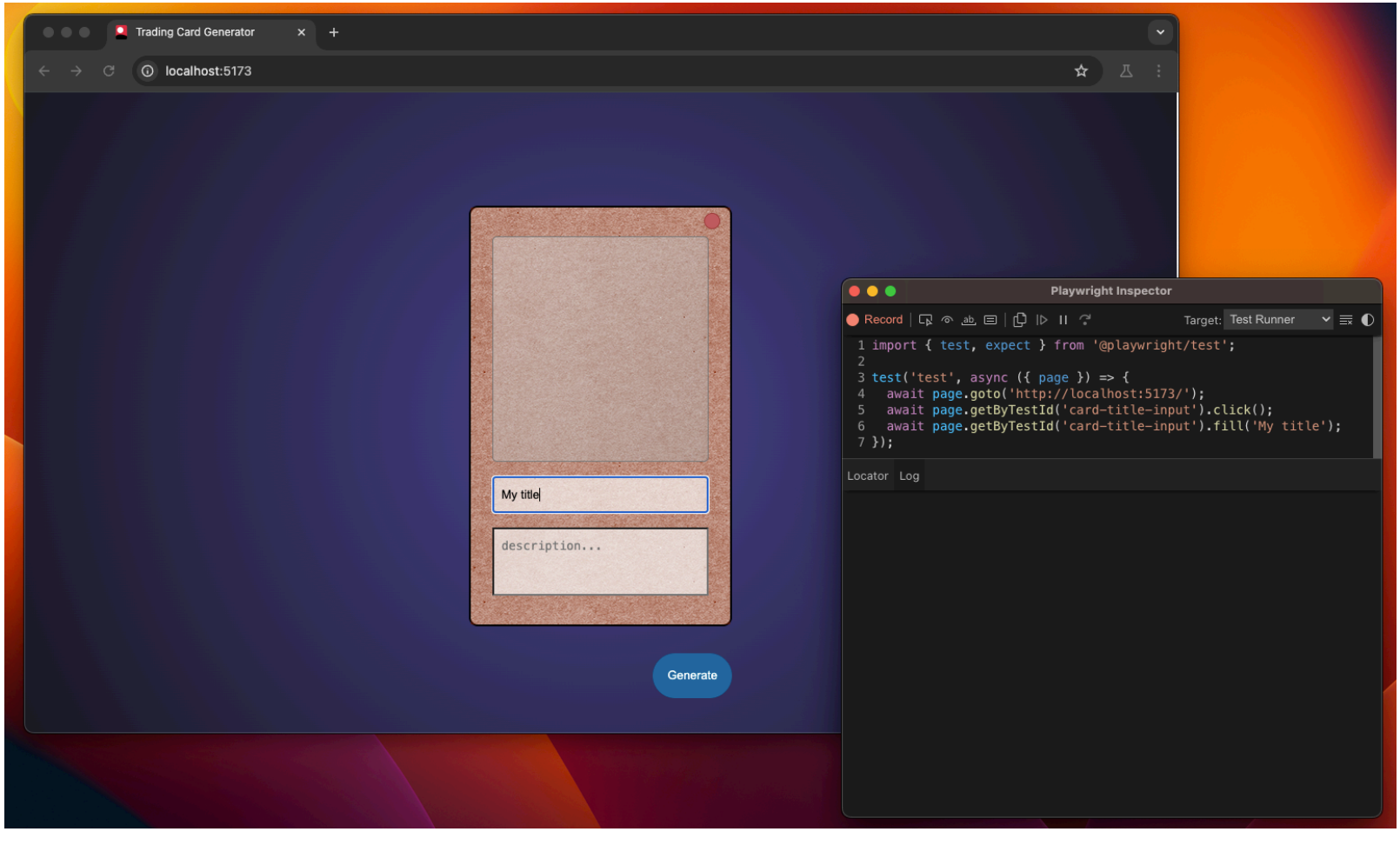
https://playwright.dev/docs/intro

```
npm init playwright@latest
```

```
npx playwright
```

PLAYWRIGHT CODEGEN

```
npx playwright codegen
```



PLAYWRIGHT EXERCISE

Let's implement an e2e test that tests our card generation logic

`exercises/04-e2e`

Git commit: `4accb2f`

Q & A

FEEDBACK

We'd appreciate you giving us any feedback you have on our workshop at this linked Google Form